FluAssist

Step-by-Step Implementation Guide

Project: SLM-Based Clinical Decision Support System for Multi-Disease Febrile Classification

Supported diseases: Dengue, Malaria, Typhoid, and Chikungunya

Primary users: Doctors and healthcare professionals

This guide converts the current project concept into a practical development plan for a beginner team. The project presentation defines the flow as doctor clinical notes -> SLM feature extraction -> preprocessing -> Random Forest classification -> confidence threshold -> disease or unknown/uncertain case -> doctor dashboard.

Important: this is a clinical decision-support prototype. It should not be presented as an autonomous diagnostic system. Final diagnosis and treatment remain with qualified clinicians and appropriate confirmatory testing.

1. Understand the System Before Coding

Your system has two different AI jobs:

A. Small Language Model (SLM): reads a doctor's free-text note and extracts structured clinical features.

B. Machine Learning classifier: uses those structured features to classify the case into one of four supported disease classes.

```
Doctor note
    ↓
SLM
    ↓
Structured features
    ↓
Preprocessing
    ↓
Random Forest / XGBoost
    ↓
Class probabilities
    ↓
Confidence threshold
    ■■■ Supported disease
    ■■■ Unknown / uncertain case
```

2. Development Rule: Build in Stages

Do not start with the SLM or the web interface. Build and verify the ML pipeline first. This makes debugging much easier.

• Stage 1: Dataset analysis and cleaning

• Stage 2: Unified four-disease dataset

• Stage 3: Random Forest baseline

• Stage 4: Model evaluation and comparison

• Stage 5: Confidence-based rejection

• Stage 6: SLM clinical-note extraction

• Stage 7: SLM-to-ML integration

• Stage 8: Streamlit doctor interface

• Stage 9: End-to-end testing and documentation

3. Step 1 - Set Up Python

Install Python 3.x and VS Code. Add the Jupyter extension in VS Code so that you can experiment in notebooks.

Create the project folder:

```
FluAssist/
■■■ datasets/
■■■ notebooks/
■■■ preprocessing/
■■■ models/
■■■ slm/
■■■ app/
■■■ requirements.txt
```

Install the initial libraries:

```
pip install pandas numpy scikit-learn matplotlib seaborn openpyxl joblib streamlit
```

Do not install the SLM stack yet. First complete the tabular ML pipeline.

4. Step 2 - Collect and Inspect the Four Datasets

Use the current datasets for Dengue, Malaria, Typhoid, and Chikungunya. Before merging anything, inspect every file and understand what each row and column means.

For each file, record:

• number of rows and columns

• column names and meanings

• target/diagnosis field

• missing-value rate

• whether the records are real, synthetic, surveillance, or severity-focused

• whether a row represents a confirmed disease case or a mixed population

Example inspection code:

```
import pandas as pd
df = pd.read_csv('dataset.csv')
print(df.shape)
print(df.columns.tolist())
print(df.head())
print(df.isnull().sum())
```

5. Step 3 - Define the Common Feature Schema

The four source datasets do not use identical columns. Create one master clinical schema and map all datasets into it.

Recommended initial master features: Age, Gender, Fever, Fever_Duration, Temperature, Headache, Chills, Fatigue, Muscle_Pain, Joint_Pain, Nausea, Vomiting, Diarrhea, Abdominal_Pain, Rash, Cough, Eye_Pain, Conjunctivitis, Platelet_Count, WBC_Count.

Do not keep a feature merely because it exists in one dataset. Retain a feature when it is clinically relevant and can be handled consistently across datasets.

6. Step 4 - Standardize Column Names and Values

Different datasets may use different names for the same clinical concept. Create a mapping layer.

Original field	Standard field
MIALGIA	Muscle_Pain
Myalgia	Muscle_Pain
Body Pain	Muscle_Pain
CEFALEIA	Headache
ARTRALGIA / Joint Pains	Joint_Pain
VOMITO / Vomitting	Vomiting
NAUSEA	Nausea
EXANTEMA / Maculopapular rash	Rash

Convert yes/no symptom values consistently, for example Yes = 1 and No = 0. But **missing information is not the same as No symptom**. Preserve missingness where the source dataset does not establish absence.

7. Step 5 - Define the Disease Label

The final training data should have exactly one target column:

```
Disease ∈ {Dengue, Malaria, Typhoid, Chikungunya}
```

For the Chikungunya file you uploaded, the field **Severe Chikungunya** is a severity-related field. If the file contains confirmed Chikungunya cases, do not use that severity field as the four-class target. Instead create Disease = Chikungunya and treat severity as outside the primary disease classifier.

8. Step 6 - Clean and Merge the Data

For each disease dataset:

• remove identifiers, administrative codes, dates, and other non-clinical fields that can create leakage

• rename columns into the common schema

• convert categorical symptom values into consistent codes

• handle missing values using a documented strategy

• add the disease label

• keep only the common/approved feature set

Then concatenate the four cleaned tables into one file:

`combined_febrile_dataset.csv`

9. Step 7 - Check Data Quality

Before training, calculate:

• total records

• records per disease

• missing values per feature

• duplicate rows

• class balance

• obviously impossible values

• whether the same patient appears multiple times in the data

```
print(df['Disease'].value_counts())
print(df.isnull().mean().sort_values(ascending=False))
```

10. Step 8 - Split the Dataset

Separate predictors X from the target y, then create training and test sets. Use a stratified split so each disease class is represented in both sets.

```
from sklearn.model_selection import train_test_split
X = df[feature_columns]
y = df['Disease']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

If you perform imputation or scaling, fit preprocessing only on the training data and apply the learned transformation to the test data. This prevents data leakage.

11. Step 9 - Train Your First ML Model

Start with Random Forest because your primary data are structured clinical/tabular features.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)
model.fit(X_train, y_train)
```

Do not claim a target accuracy before you train the model. The actual result depends on dataset quality, class balance, preprocessing, and leakage control.

12. Step 10 - Evaluate Properly

Report more than accuracy. Use:

• Accuracy

• Precision

• Recall

• F1-score

• Confusion matrix

• Per-class performance

```
from sklearn.metrics import classification_report, confusion_matrix
pred = model.predict(X_test)
print(classification_report(y_test, pred))
print(confusion_matrix(y_test, pred))
```

13. Step 11 - Compare Random Forest and XGBoost

Your methodology lists Random Forest/XGBoost. Implement Random Forest first, then train XGBoost as a benchmark. Compare them using the same train/test split and the same feature set.

Choose the final classifier based on measured performance and practical considerations. In the final report, state which model was selected and why.

14. Step 12 - Save the Final Model

```
import joblib
joblib.dump(model, 'models/febrile_classifier.pkl')
```

Also save the exact feature order, preprocessing objects, and category mappings. The application must use exactly the same feature order used during training.

15. Step 13 - Build a Manual Prediction Form First

Before the SLM, build a simple form that lets the developer or doctor enter structured values manually.

```
Fever: Yes
Headache: Yes
Chills: No
Muscle Pain: Yes
Joint Pain: Yes
Vomiting: No
Rash: No
Platelet Count: 80000

[PREDICT]
```

If this produces a sensible model prediction, the ML part is functioning independently of the SLM.

16. Step 14 - Add Prediction Probabilities

```
probabilities = model.predict_proba(X_input)[0]
```

Map those probabilities to the four disease names and identify the highest class probability.

17. Step 15 - Add the Unknown/Uncertain Case Rule

A classifier trained on four classes will always prefer one of those classes. Your threshold is a rejection mechanism. It does not discover the true outside disease.

```
max_prob = probabilities.max()
if max_prob >= threshold:
    result = predicted_class
else:
    result = 'Unknown / Uncertain Case'
```

Select the threshold using validation data rather than arbitrarily declaring 70%, 80%, or 90%. Later, investigate probability calibration if required.

18. Step 16 - Choose and Run the SLM

Only after the ML pipeline works should you add the SLM. Your current project plan lists lightweight models such as Phi-3 or Qwen.

> The SLM does not decide the disease. Its role is to read the doctor's note and extract the same features used by the ML model.

19. Step 17 - Design the SLM Extraction Prompt

```
Extract only the approved clinical features from the note.
Return valid JSON only.
Use null when the note does not contain the information.

Features:
fever, fever_duration, headache, chills, fatigue,
muscle_pain, joint_pain, nausea, vomiting, diarrhea,
abdominal_pain, rash, cough, eye_pain, conjunctivitis,
platelet_count, wbc_count
```

Example doctor note:

“Patient has high fever for four days with severe headache, chills, generalized body pain, vomiting and severe joint pain. Platelet count is 80,000/mm³.”

Expected structured output:

```
{
  "fever": 1,
  "fever_duration": 4,
  "headache": 1,
  "chills": 1,
  "muscle_pain": 1,
  "joint_pain": 1,
  "vomiting": 1,
  "platelet_count": 80000
}
```

20. Step 18 - Validate SLM Extraction

Do not automatically trust the SLM output. Show extracted fields to the doctor for confirmation or correction before prediction.

```
SLM extracted:
✓ Fever
✓ Headache
✓ Chills
✓ Muscle Pain
✓ Joint Pain
✓ Vomiting
Platelets: 80,000

[Confirm / Edit]
```

> This manual confirmation step is an important safety and usability feature because an SLM can misread negation, quantities, or context.

21. Step 19 - Connect SLM to the ML Model

```
Doctor note
   ↓
SLM JSON extraction
   ↓
Feature validation/edit
   ↓
Common feature schema
   ↓
ML preprocessing
   ↓
Random Forest / XGBoost
   ↓
Class probabilities
   ↓
Threshold decision
```

At this stage, the system becomes the hybrid AI system described in your proposal.

22. Step 20 - Add Clinical Action Suggestions Carefully

After a high-confidence classification, the interface may show a reminder about appropriate confirmatory testing. Keep these as decision-support prompts, not automatic treatment instructions.

Predicted class	Example investigation reminder
Dengue	Appropriate dengue laboratory confirmation
Malaria	Rapid diagnostic test and/or peripheral smear
Typhoid	Appropriate microbiological confirmation
Chikungunya	Appropriate Chikungunya laboratory confirmation

23. Step 21 - Build the Streamlit Doctor Dashboard

Create a simple functional interface first:

```
FLUASSIST

Doctor's Clinical Note
[_____________________________]

[Extract Clinical Features]

Extracted Features
[editable fields]

[Confirm & Predict]

Prediction: Dengue
Probability: 0.91

Action: Consider confirmatory testing / clinical evaluation
```

Do not spend early development time on advanced UI styling. Functionality and correctness are the priorities.

24. Step 22 - Test the System in Layers

• Dataset tests - verify columns, labels, class balance, missingness, and leakage.

• ML tests - evaluate accuracy, precision, recall, F1, and confusion matrix.

• Probability tests - verify probability outputs and threshold logic.

• SLM tests - evaluate whether each clinical feature is extracted correctly.

• Integration tests - ensure the SLM output is mapped correctly into the ML schema.

• UI tests - verify that invalid or incomplete inputs do not crash the application.

• Clinical safety tests - verify that the UI consistently states that the tool is decision support and not a final diagnosis.

25. Example End-to-End Test

Input: “Patient has fever for four days, severe headache, chills, body pain, vomiting and joint pain.”

SLM output: structured symptoms and duration.

ML output: four class probabilities.

Decision: supported disease if the maximum probability meets the validated threshold; otherwise Unknown/Uncertain Case.

Final screen: disease/probability, optional confirmatory-test reminder, and a statement that the clinician retains final responsibility.

26. Team Division for Four Members

Member	Main responsibility
Member 1	Dataset collection, cleaning, feature mapping, merged dataset
Member 2	ML model, evaluation, confusion matrix, model saving
Member 3	SLM selection, prompt design, extraction, validation
Member 4	Streamlit UI, integration, threshold logic, testing

All four members should still understand the complete pipeline for the final viva.

27. What to Do First - The Immediate 7-Day Plan

Day	Task	Deliverable
1	Inspect all four datasets	Column dictionary + record counts
2	Select common clinical features	Approved master schema
3	Map and clean each dataset	Four standardized tables
4	Merge datasets and check quality	combined_febrile_dataset.csv
5	Train Random Forest	First working classifier
6	Evaluate and compare with XGBoost	Metrics + confusion matrix
7	Build manual predictor	Working ML-only prototype

After Day 7, you should have a functioning four-disease ML predictor. Only then should you begin the SLM integration.

28. Common Mistakes to Avoid

• Training on patient IDs, dates, hospital codes, or administrative fields.

• Using the target/diagnosis field as an input feature.

• Treating every missing value as a negative symptom.

• Merging datasets without standardizing clinical definitions.

• Claiming that the model can identify any disease outside the four classes.

• Calling raw model probability a guaranteed clinical certainty.

• Testing on data that leaked into training.

• Starting with the UI before validating the model.

• Reporting invented accuracy numbers.

• Allowing the SLM to directly decide the disease without the ML classifier.

29. Final Architecture

```
Doctor
  ↓
Clinical Notes + Structured Findings + Available Lab Values
  ↓
Small Language Model
  ↓
Clinical Feature Extraction
  ↓
Doctor Confirmation / Edit
  ↓
Feature Preprocessing
  ↓
Random Forest / XGBoost
  ↓
Disease Probabilities
  ↓
Confidence Threshold
  ■■■ Dengue / Malaria / Typhoid / Chikungunya
  ■■■ Unknown / Uncertain Case
  ↓
Doctor Dashboard + Clinical Follow-up Reminder
```

30. Final Milestone Checklist

■ Unified four-disease dataset created and documented.

■ No patient identifiers or target leakage in the feature set.

- ■ Baseline ML model trained and evaluated.
- ■ Random Forest and XGBoost compared fairly.
- ■ Final model saved with preprocessing artifacts.
- ■ Confidence threshold selected using validation data.
- ■ SLM extracts approved clinical features in a consistent structured format.
- ■ Doctor can review/correct extracted features.
- ■ SLM output successfully feeds the ML model.
- ■ Streamlit dashboard works end-to-end.
- ■ Unknown/uncertain cases are handled correctly.
- ■ Evaluation results and limitations are documented.

Recommended starting point: begin with dataset analysis and the unified feature schema. Do not start with the SLM. Your first concrete deliverable should be a clean four-disease dataset and a basic Random Forest classifier that can be evaluated independently.